

AI Usage Journal

CSE 341 Semester Project — Recipix v4.1 Parser

15 entries across 6 Claude sessions, May 6–8 2026

Overview

Six Claude sessions over three days, in three roles. Two Claude.ai web sessions on May 6 covered the design-stage brainstorming — first surveying candidate DSL domains, then drilling into the Recipix concept once we had picked it. Two Claude Code PM sessions on May 7 (Sonnet 4.6 then Opus 4.7) supervised the implementation; two autonomous-agent sessions (developer agent + tester agent, both Sonnet 4.6) produced and stress-tested the parser. The web and PM sessions were short, conversational, and opinion-driven; the agent sessions each ran in a single long turn with many tool calls. The journal entries below preserve that distinction.

Session 1 — Web Brainstorming A: Domain Survey (Claude.ai Web)

Source: claude.ai/share/4b1d48f5-7556-4559-b0fa-91080efa8085

A Claude.ai web chat used for general brainstorming over candidate DSL domains for the project, before my partner and I had committed to one. Run two days before any code was written. One substantive session.

Entry # 1 of 15 • Date 2026-05-06 • Phase Design / domain selection • AI tool Claude.ai web (Sonnet)

Goal.

Use a Claude.ai web chat to think through candidate DSL domains for the CSE 341 project before committing to one with my partner.

Prompt.

Pasted the project handout and walked Claude through the seven sample domains in §2 (TurtleLang, ChessDSL, QueryLite, MusicScript, CircuitLang, RecipeLang, PlotDSL) plus a couple of my own ideas. Asked it to compare them along three axes: originality risk vs. classmate overlap, expression-language richness for the §5 rubric’s “design justification” dimension, and rough Part 2 implementation cost.

Accepted.

The three-axis framing (originality / expression richness / Part 2 cost) became the language I used in the partner conversation that picked the domain. Also kept the observation that turtle- and circuit-style DSLs make precedence/associativity discussion harder because their natural expression languages are flat — that pushed me away from those candidates without me having to argue from instinct.

Rejected / modified.

Did not commit to a domain in this chat — held the decision for a partner conversation, which Claude correctly suggested rather than pushing for a pick. I also rejected its first-pass framing of “pick whichever is easiest to implement”: the §5 rubric weighs design justification and originality much more heavily than implementation breadth, and going with the easiest option would have starved the exam of material.

Errors caught.

No code in this session, so no implementation bugs. One soft error: when I asked for an off-the-cuff originality ranking of the seven sample domains, Claude gave a confident ordering it could not really defend on a re-read — I treated that ranking as opinion rather than data and asked it to justify each placement before using any of it.

Reflection.

Web-chat brainstorming is best at structuring my own thinking, not at making the decision for me. The three-axis framing was the only artefact I carried into the partner conversation; everything else was just me hearing my options out loud. Worth doing — but next time I’ll ask for the framing first rather than burning prose on each candidate before knowing which axes I care about.

Session 2 — Web Brainstorming B: Recipix Concept (Claude.ai Web)

Source: claude.ai/share/e087c600-defd-4c38-af23-396b739c9a57

A second Claude.ai web chat run later the same day, after my partner and I had narrowed to “recipe DSL with dimensional types.” Used to stress-test the concept before committing to a written spec. One substantive session.

Entry # 2 of 15 • Date 2026-05-06 • Phase Design / Recipix concept stress-test • AI tool Claude.ai web (Sonnet)

Goal.

Stress-test the “recipe DSL with dimensional types” concept end-to-end before any spec was written: figure out which design decisions would carry the originality and exam load, and which were nice-to-have.

Prompt.

Sketched the rough idea — a recipe language with quantity literals, ingredient declarations, scale and substitute as domain operators, and a dimensional type system over Mass / Volume / Count / Temperature / Duration — and asked Claude to argue both for and against it as a CSE 341 project domain. Then iterated through follow-up questions on each component (record vs. ad-hoc structured type, where scale/substitute should live, what equivalence rule the recipe type should use).

Accepted.

Kept the “dimensional types as the originality anchor” framing — it gave Recipix something genuinely different from a generic scripting language and gave us Sebesta §6.5–6.7 (structured-type design decisions) to defend at the exam. Also kept the suggestion that scale and substitute should be top-level call-site constructs rather than inside-recipe statements; that became load-bearing the next day, locked in the v4.1 spec as Decisions #34 and #35.

Rejected / modified.

Rejected an early suggestion to make recipes first-class values you can pass around: would have ballooned the scope into closures and added no exam-defensible material. Also pushed back on Claude's tendency to want every operator to coerce — keeping coercion narrow (int → float, never int → quantity, never across dimensions) was deliberate, and Claude only acknowledged the trade-off after I named it explicitly.

Errors caught.

Two real catches. (1) Claude suggested using structural equivalence for the recipe type “for ergonomic reasons,” which would have collided with the Sebesta §6.14 question we knew the exam would ask — we kept name equivalence instead, and the May 7 professor session later confirmed the choice. (2) It floated “no need for control structures, recipes are declarative,” which directly violates handout §2's minimum requirement of at least one control structure beyond sequencing. Both errors detected by re-reading the handout against its claims rather than trusting the prose.

Reflection.

The single most useful thing this chat did was force me to articulate why this language had to exist — what would be ungainly to express in plain Python or a spreadsheet. The dimensional type system answered that question, and the answer survived four spec-review passes the next day. Web-chat brainstorming is the right tool for design rationale; it is the wrong tool for the spec itself, which has to live in a file rather than scattered across chat history.

Session 3 — PM Session A (Claude Sonnet 4.6)

Source session: `6dc29a8d-d603-4ee8-9780-f6d26f533269.jsonl`

First attempt at running a project-manager session. Asked Claude to act as PM, read the course materials, and orchestrate a developer agent. Quickly realized the model was about to dispatch the developer itself, which I didn't want. Cut this session short and restarted with Session 4.

Entry # 3 of 15 • Date 2026-05-07 • Phase Project setup / PM brief • AI tool Claude Sonnet 4.6

Goal.

Get the AI fully briefed on the Recipix project (course handout, submission guide, v4.1 language spec) and have it adopt a PM-only role for Part 1 (lexer + parser).

Prompt.

Asked it to read `CSE341_Project_Handout.pdf`, the submission guide, and `recipix_v4_1_spec.md`, then act as a project manager who would supervise a developer agent. Stressed: “you are the project manager and I want you to solely focus on the topic and nothing else.” Attached the two PDFs.

Accepted.

It read all three documents using Glob/Read tools and produced a clean briefing: a deliverables table (D1–D6), correct identification of the deadline (May 8, 23:59), the right scope (D2 = lexer + parser), and a sharp risk list of spec corners that AI parsers usually slip on (Decisions #7, #17,

#20, #23, #34, #35). It also asked the right clarifying questions before kicking off (Python version, whether D1 was already drafted).

Rejected / modified.

The PM tone defaulted to a more "I will run the developer agent" framing than I wanted. It put itself in the executor seat, not just the reviewer seat. I had to push back later in the session to clarify I would drive the agents.

Errors caught.

Caught no factual errors in the briefing — the deliverables table and rubric percentages matched the handout. The risk list correctly identified that quantity literals are two tokens (Decision #7), which the developer agent would later trip on if not pre-warned.

Reflection.

Briefing-from-spec is one of the things AI is genuinely good at. The dense per-decision risk list saved me real time — half of those risks did materialize in later rounds. Lesson: when dispatching to a sub-agent, lock down which side of the table the PM AI is on (reviewer vs. executor) in the very first message, not the third.

Entry # 4 of 15 • Date 2026-05-07 • Phase Course-correction • AI tool Claude Sonnet 4.6

Goal.

Remind it that the design spec already exists (in the .md it just read) and let it proceed.

Prompt.

"design spec is in the md file I told you to read."

Accepted.

It acknowledged the spec was locked and said "Dispatching the developer agent now."

Rejected / modified.

It immediately invoked the Agent tool to launch a sub-agent without giving me a reviewable prompt first. This is exactly what I didn't want — I had asked for PM-only behavior. I interrupted the tool call.

Errors caught.

Workflow error rather than a code error: the model collapsed PM-and-executor into a single turn. The interrupt cost a few seconds but no real damage.

Reflection.

Single-line nudges are too low-bandwidth for a model that has just been given a PM role with implicit "make progress" pressure. Should have replied with a one-line constraint ("PM only — give me the prompt, do not run it") instead of just confirming the spec source.

Entry # 5 of 15 • Date 2026-05-07 • Phase Developer-agent prompt drafting • AI tool Claude Sonnet 4.6

Goal.

Get a copy-pasteable prompt for the developer agent that builds only the parser (the lexer is owned by my partner).

Prompt.

"design spec is in the md file I told you to read. We are going to build only the parser. Lexer is being built [by my] friend. Give me the prompts for developer agent and I will handle him myself. Don't run it yourself."

Accepted.

Produced a ~16k-character developer-agent prompt: scope (parser only, Python 3.11+, stdlib only), the assumed Token/TokenType contract from the lexer team, mandatory AST shapes pulled verbatim from spec §14 (QuantityOf, SubstituteCall with str slots), per-decision implementation traps, file layout, and acceptance criteria. The §14 AST quote was correct.

Rejected / modified.

Length-wise the prompt was on the edge of overwhelming — useful, but I trimmed nothing because I wasn't sure which trap would matter. In hindsight I should have kept a shorter master prompt and put trap details in an appendix the agent reads on demand.

Errors caught.

Caught no errors in the produced prompt. Cross-referenced two of the §14 dataclass quotes against the spec — they matched.

Reflection.

Once the role boundary was clear ("PM only, you give me prompts"), the output was exactly what I needed. Pre-loading per-decision traps into the developer prompt is the right call: it costs a few hundred tokens up front and saves a review round later. I closed this session here and switched models for the next attempt to compare Sonnet vs Opus on the same task.

Session 4 — PM Session B (Claude Opus 4.7)

Source session: ba1b28db-29e3-45a8-97f5-afb7a6c21561.jsonl

Restarted the PM session with Opus 4.7 to compare against the Sonnet 4.6 attempt. This became the main PM thread — used across two days for the developer prompt, the post-developer review, the bug-fix round, and final triage. Seven substantive turns.

Entry # 6 of 15 • Date 2026-05-07 • Phase Project setup / PM brief • AI tool Claude Opus 4.7

Goal.

Same as entry 3, but with Opus instead of Sonnet — get a PM brief and confirm the model can handle the multi-day arc.

Prompt.

Same as entry 3, attached the same two PDFs.

Accepted.

The brief was tighter than Sonnet's. It correctly narrowed scope to D2 only (lexer + parser, no type checker / interpreter — those are Part 2), produced an identical risk list of 10 spec corners, but also explicitly called out two additional traps Sonnet had missed: that the `serves <expr>` parse must not swallow the body brace, and that all 12 action verbs are reserved keywords. It did not auto-dispatch — it ended with two clarifying questions (Python OK? proposed repo layout OK?) and waited.

Rejected / modified.

Nothing rejected. The proposed repo layout (`src/recipix/{lexer.py, parser.py, ast\_nodes.py, errors.py, cli.py}`) became the actual layout we shipped.

Errors caught.

Caught no errors. Risk list matched the spec.

Reflection.

Opus's behaviour here was the better of the two PM models: it stopped and asked, instead of acting. The two extra traps it surfaced (serves-greedy, action-verbs-as-keywords) both showed up later as real edge cases. Continuing this thread.

Entry # 7 of 15 • Date 2026-05-07 • Phase Developer-agent prompt drafting • AI tool Claude Opus 4.7

Goal.

Get the developer-agent prompt I'll paste into a separate session.

Prompt.

"Python is okay yes. Don't run the developer agent yourself give me the prompts and I will run the developer and tester agents... You are the PM only. Lexer is being built by my friend. Our job is to build the parser."

Accepted.

The prompt was self-contained, front-loaded the spec anchors and the trap list, included a "Token contract" section so the developer would publish a stable interface for the lexer team, and ended with a "do not mark complete unless every spec corner is verified" rule. It also wrote the prompt to disk via Write tool so I had a clean file (not just a chat scrollback).

Rejected / modified.

Nothing rejected. Took it as-is.

Errors caught.

None.

Reflection.

The Write-to-disk move was a nice touch — once the PM thread is long, scrolling back to find the original developer prompt is annoying. Saving it as a file from the get-go made paste-into-new-agent trivial. Will keep using this pattern.

Entry # 8 of 15 • Date 2026-05-07 • Phase Post-developer review • AI tool Claude Opus 4.7

Goal.

Have the PM read the developer agent's output, run the tests, and produce a realistic review (not a rubber stamp).

Prompt.

"I want you to go over and create a realistic review of developer's job."

Accepted.

Read the parser, ran the test suite (48/48 green), and probed 12 edge cases the existing tests didn't cover. Verdict was nuanced: ✅ ship-quality with three deferred-issues to confirm and two minor fixes recommended. The deferred items (early-return enforcement, scale/substitute call-site-only enforcement, zero-arg call ambiguity) were each defensible and pointed to the right pipeline phase. The two minor fixes (delete duplicate tokens.py, reject trailing commas in list literals) were both real.

Rejected / modified.

Nothing rejected — every claim it made checked out against the parser source on a spot-check.

Errors caught.

Caught a real silent-failure mode: the developer had created `src/ecipix/tokens.py` while the lexer team had `src/tokens.py`, with two parallel Token classes that duck-typed compatibly today but would silently break the day either side renamed a constant. I had not noticed this. The "trailing comma silently accepted in list literals" finding was also real — neither the developer nor I had thought about it.

Reflection.

This is the entry where the AI most clearly earned its keep. The "two parallel tokens.py" finding was the kind of thing I would have caught only after a future bug, not before submission. Probing 12 edge cases beyond the existing tests is what a good code reviewer does and what I can't reliably motivate myself to do at 2am. Lesson: explicitly ask for "realistic" or "honest" review — without the word, the default is closer to a rubber stamp.

Entry # 9 of 15 • Date 2026-05-08 • Phase Status check + tester-agent prompt • AI tool Claude Opus 4.7

Goal.

Check what changed since the last review (my partner had done bug fixing), figure out what's still outstanding, and produce a tester-agent prompt that hunts for real bugs.

Prompt.

"my friend did some bug fixing. Check the current status of the project. What else needs to be done. Give me a prompt for me to create a tester agent whose job is to learn the language and test it to find bugs and issues. And to create me a report"

Accepted.

Diffed against its previous review using Bash/git. Confirmed Revision 1 was done (single tokens.py, lexer's Decision #7 fix landed, all 48 parser tests still green, CLI smoke test on sample1.rcx clean). Confirmed Revision 2 was still outstanding (trailing commas still accepted).

Then produced a tester-agent prompt with a two-phase structure (Phase 1: learn the language, Phase 2: hunt bugs) and 15 mandatory test categories pinned to specific spec decisions.

Rejected / modified.

Nothing rejected. The "15 mandatory categories" framing was load-bearing — without it the tester agent would have wandered.

Errors caught.

Verified its claims by re-running the tests myself; matched.

Reflection.

Resumability — the model picking up its own previous review and diffing only against it — is something a multi-day PM workflow needs. This worked because the review in entry 8 had been written down with concrete file paths and line numbers, so the model could literally re-read its own output and check each item. Lesson: when asking for review output, demand it cite file:line, even though it costs tokens — it pays for itself the moment the next round starts.

Entry # 10 of 15 • Date 2026-05-08 • Phase Triage + bug-fix prompt • AI tool Claude Opus 4.7

Goal.

Triage the BUG_REPORT.md the tester agent produced and produce a focused bug-fix prompt for the developer agent.

Prompt.

"test agent is done it created a BUG_REPORT.md create the developer agent prompt to fix the bugs."

Accepted.

Read the bug report, separated parser bugs from lexer bugs (correctly identified Bug 8 as a lexer bug — not the parser developer's responsibility). Made an explicit PM ruling on Gap 1 (the spec example used /* */ which the spec elsewhere doesn't define — ruled it a typo). Produced a Round-2 prompt that listed exactly which 8 bugs + 1 gap to fix, with discipline rules (don't refactor, don't change AST shapes, don't change error message format).

Rejected / modified.

Nothing rejected. The triage step (explicitly classifying which bugs the parser developer should NOT touch) was correct.

Errors caught.

Did not catch any of its own errors but the framing of "don't change AST node shapes" was important — this prevented the developer agent from drifting in Round 2. Tested by reading the next session's output (entry 15): no AST changes happened.

Reflection.

The single most valuable PM behaviour here was the "NOT for parser developer" list — without that, the developer agent would have gone hunting for Bug 8 (lexer) and made things worse. AI is bad at owning team boundaries unless you explicitly draw them.

Entry # 11 of 15 • Date 2026-05-08 • Phase Final readiness + Q&A • AI tool Claude Opus 4.7

Goal.

Quick yes/no check on whether the project is submission-ready, plus answer a confused question about the tester agent's output.

Prompt.

"developer agent is done. One question why in the world did tester agent created a test_lexer and test_parser code and not use the original codes? Be very brief with your answers. Also is this work good enough to submit for part 1 again be very brief with your answer."

Accepted.

It corrected my misread directly: the tester agent had NOT created test_lexer.py / test_parser.py — those were the originals; the tester only created `rcx` fixture files under tests/tester/. Then a clean yes for D2 readiness with the verification numbers (69/69 tests green, 8 bug fixes verified, lexer/parser cleanly separated, line-numbered errors). Closed with what I still owe (D1 PDF, D3 PDF, D4 journal, D5 retro, D6 contribution report).

Rejected / modified.

Nothing rejected.

Errors caught.

Caught my own factual error — I had misattributed the test_lexer.py / test_parser.py files to the tester agent. The model didn't hedge or flatter; it said "It didn't" directly.

Reflection.

I asked it to be brief and it was. When the human is wrong, the AI saying "no, here's what actually happened" is more useful than diplomatic equivocation. This was a small turn but it stopped me from going to interrogate the tester agent for nothing.

Entry # 12 of 15 • Date 2026-05-08 • Phase Final readiness + Q&A • AI tool Claude Opus 4.7

Goal.

Push back on the previous answer because I was looking at test_lexer.py / test_parser.py with my own eyes and didn't believe the "they're the originals" answer.

Prompt.

"there are 2 files named test_lexer.py and test_parser.py under test folder. I am literally looking at them. What are they for? Why would tester agent create different parser and lexer to test the main project. be brief with your answer"

Accepted.

Used Bash + git log to verify and answered with provenance: test_lexer.py is from my partner's commit d9988fc ("Lexer done"), test_parser.py is from the original developer agent's commit 4a7a638 ("parser"). Confirmed both files import the real lexer and parser (no shadow implementations) — they're unit tests against the real code, not alternative implementations.

Rejected / modified.

Nothing rejected.

Errors caught.

Caught my false-alarm. I had assumed any unfamiliar file under tests/ was tester-agent-generated, which was wrong.

Reflection.

Insistence-with-evidence beats insistence-alone. The model didn't cave when I doubled down — it ran `git log` and produced commit hashes. Lesson for me: when I "literally see" something that contradicts the model, that's the right time to ask it to verify, not to assume it's wrong.

Session 5 — Tester Agent (Claude Sonnet 4.6)

Source session: *e88d3ad2-4454-4c3e-80db-e1b72a133139.jsonl*

A fresh session running the tester-agent prompt that the PM (Session 4, entry 9) produced. Single user message, 119 tool calls of autonomous work that produced ~70 fixture programs and a BUG_REPORT.md with 9 findings. Treated as one journal entry for the whole agent run.

Entry # 13 of 15 • Date 2026-05-08 • Phase Testing / bug-hunting (autonomous agent) • AI tool Claude Sonnet 4.6

Goal.

Pasted the PM-supplied tester-agent prompt: learn the Recipix language by reading the spec and writing programs in it (Phase 1), then try to break the lexer + parser and produce a written bug report (Phase 2). 15 mandatory test categories tied to specific spec decisions.

Prompt.

Pasted the ~8.4k-character tester-agent prompt from the PM session verbatim — included Phase 1/2 structure, the 15 mandatory categories, "find bugs not fix bugs" discipline, and instructions to file BUG_REPORT.md.

Accepted.

It executed the full agentic loop: read the spec end-to-end, ran the three sample programs through the CLI as a sanity check, examined the parser and AST source, wrote ~70 .rcx fixture files under tests/tester/ (split into should_pass / should_fail), ran each one, and produced a categorized BUG_REPORT.md with concrete findings. Tools used: 31 Bash, 70 Write, 10 Read, 4 Glob, 1 Edit, 3 chapter-marker calls. The bug report covered: comparison non-associativity edge cases, scale/substitute placement leaks into recipe bodies, missing recursion-limit handling, duplicate at/for modifier detection, substitute slot error-message clarity, quantity_of empty-operand handling, if-statement error line attribution, and a list-literal trailing-comma gap.

Rejected / modified.

The agent flagged a few "edge cases" that on review weren't actually spec violations — the PM (Session 4 entry 10) later filtered these out. Specifically, Bug 8 turned out to be a lexer bug (out of scope for this round) and Gap 1 turned out to be a typo in the spec example, not a real bug in the parser. Both were correctly raised by the tester for the PM to triage; the tester did not pretend to know which bucket they belonged in.

Errors caught.

The fixtures themselves did the catching. Each fixture is a program that either should pass or should fail; running them surfaced 8 real parser bugs + 1 spec gap. The most surprising find was Bug 1: the "non-associative comparison" rule was over-eager — ``a < b == c`` was rejected even though it should parse as ``(a < b) == c``. The non-associativity rule had been written too broadly.

Reflection.

The autonomous tester pattern worked because the prompt over-specified the categories. With 15 mandatory categories pinned to spec decisions, the agent couldn't drift into uninteresting fuzz; every fixture had a job. AI is good at the volume side of testing — writing 70 small focused programs is tedious for a human and not error-prone for a model. The trade-off: 8 of the findings were real, but ~3 of the flagged "edges" were noise the PM had to filter. Net: still a big win in time.

Session 6 — Developer Agent (Claude Sonnet 4.6)

Source session: *f0e44215-c396-46ee-b695-93a6a67be9d4.jsonl*

Two-round developer-agent session: Round 1 built the parser from scratch using the PM's prompt; Round 2 did the bug-fix pass after the tester agent's report. Each round was a single long autonomous run, treated as one journal entry.

Entry # 14 of 15 • Date 2026-05-07 • Phase Parser implementation (autonomous agent) • AI tool Claude Sonnet 4.6

Goal.

Build a recursive-descent parser for Recipix v4.1 in Python — AST nodes, parser, pretty-printer, CLI, and tests. Lexer is owned by the partner team; this agent only consumes Token / TokenType through a published contract.

Prompt.

Pasted the ~7.2k-character developer-agent prompt from Session 4 entry 7: scope, repo layout, mandatory AST shapes from spec §14 (QuantityOf, SubstituteCall with str slots), the per-decision trap list (#7, #17, #20, #23, #34, #35), and acceptance criteria.

Accepted.

It produced the full parser package matching the prompted layout: `tokens.py` (the lexer-side contract), `ast_nodes.py`, `parser.py`, `errors.py`, `pretty.py`, `__main__.py`, plus three valid sample programs from spec §11, five invalid fixtures, and a unit test file. Final test count 48/48 green. The pre-committed AST shapes (QuantityOf with operand: Expr, SubstituteCall with original_name: str / replacement_name: str) were exactly as the spec dictates. The non-associative comparison rule was implemented (Decision #17). Mandatory braces on if/else were enforced (#20). Fixed step modifier order at-before-for (#23). Tools: 18 Write, 22 Read across rounds, 6 Bash for verification, 6 TodoWrite for self-tracking.

Rejected / modified.

The Decision #17 implementation was overzealous — it rejected ``a < b == c`` at the top of ``_parse_eq``, which the tester agent later flagged in Round 2 as Bug 1. So the implementation

passed all 48 of its own tests but the tests didn't cover the cross-level case. This is a "correct against your own tests, wrong against the spec" failure mode — exactly the kind the trap list is meant to prevent but in this case the trap was framed too narrowly (" $a < b < c$ must fail") and the agent generalized incorrectly.

Errors caught.

The agent self-caught: an early version of `\_parse\_eq` had unnecessarily complex op-string handling; it noticed and refactored mid-run. Did NOT self-catch the Bug 1 over-rejection; that took the tester agent to find.

Reflection.

The TodoWrite self-tracking was helpful — the agent's own todo list mirrored the prompt's acceptance criteria, and it ticked them off honestly. AI is good at breadth-first implementation when the spec is locked. AI is unreliable on second-order semantics ("the rule is X" → "the dual rule must therefore be Y"). The over-reject in Bug 1 is a textbook case: the spec says "non-associative comparisons" and the agent generalized to "comparisons cannot appear on either side of comparisons" instead of the narrower "the same comparison cannot chain". Next time: pin the rule with a *positive* example that should pass alongside the negative example that should fail.

Entry # 15 of 15 • Date 2026-05-08 • Phase Bug fixes round 2 (autonomous agent) • AI tool Claude Sonnet 4.6

Goal.

Fix the 8 parser bugs + 1 gap from BUG_REPORT.md without changing AST shapes or error message format. Add a new TestBugFixesRound2 class with one regression test per bug.

Prompt.

Pasted the ~8.4k-character bug-fix prompt from Session 4 entry 10: explicit list of bugs to fix (Bugs 1, 2, 3, 4, 5, 6, 7, 9 + Gap 2), explicit list of bugs NOT to touch (Bug 8 — lexer's problem; Gap 1 — spec typo PM ruled out), discipline rules (no refactoring, no AST changes, no error format changes), and a CLI verification checklist of 10 commands.

Accepted.

8 fixes + Gap 2 landed in three files (parser.py, __main__.py, tests/test_parser.py) plus one new invalid fixture. Test count went 48 → 69. Every CLI verification command in the prompt produced the expected output. Fixes: removed EQ/NEQ from level-4 non-assoc guard (Bug 1), added `_in_construction_body` flag for scale/substitute placement (Bug 2), setrecursionlimit + RecursionError handling (Bug 3), duplicate at/for detection (Bugs 4+5), distinguished missing-ratio: from expression-in-slot error (Bug 6), quantity_of empty-operand check (Bug 7), if-brace error attributed to the if line not the unexpected token (Bug 9), trailing-comma rejection in list literals (Gap 2).

Rejected / modified.

Two test failures appeared on the first full-suite run: `test_bug3_deep_nesting_parseable` (the `setrecursionlimit` was CLI-only, the unittest needed its own guard) and `test_chained_mixed_fails` (after Bug 1's fix, ``1 < 2 == 2`` is now valid, so the old assertion was wrong). The agent fixed both within the same run rather than ignoring them.

Errors caught.

Self-caught both test failures and resolved them. Also flagged a spec ambiguity to the PM: the Bug 1 fix uses ``isinstance(right, CompareOp)`` to detect a right-side comparison, but this also rejects ``a == (b < c)`` (parens around a comparison). The agent honestly named the limitation rather than papering over it.

Reflection.

When the prompt is "fix X exactly, don't touch anything else", AI sticks to the bound. The discipline rules in the bug-fix prompt did real work — they prevented the kind of scope creep that often happens when you tell a model "while you're in there, also clean up Y". The honest spec-ambiguity flag at the end is the single most useful thing an autonomous agent can do: name what it didn't solve, so the human knows where to look. I've started treating "did the agent flag any ambiguity?" as a quality signal in its own right.